Computer Organisation & Program Execution 2019

5

Asynchronism

Uwe R. Zimmer - The Australian National University

Asynchronism

References for this chapter

[Patterson17]
David A. Patterson & John L. Hennessy
Computer Organization and Design – The Hardware/Software Interface
Chapter 4 "The Processor",
Chapter 6 "Parallel Processors from Client to Cloud"
ARM edition, Morgan Kaufmann 2017

Asynchronism

Why?

How do you handle your communication flow?

Asynchronism

Why?

How do you handle your communication flow?

☞ Do you have times when you check certain communication?

☞ Is certain communication interrupting you? – at any time?

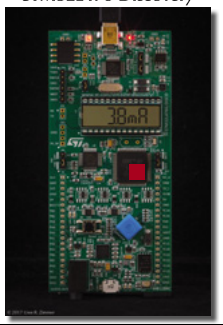
☞ Do you assign "importance levels" to your communication channels/sources?

Asynchronism

STM32L476 Discovery

CPU

… running its sequence of machine instructions.



Asynchronism

STM32L476 Discovery

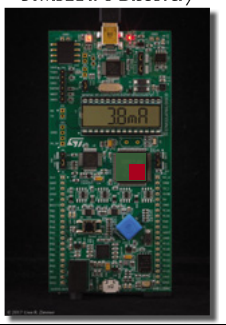
CPU

… running its sequence of machine instructions.

☞ How to interact with all the other devices inside the

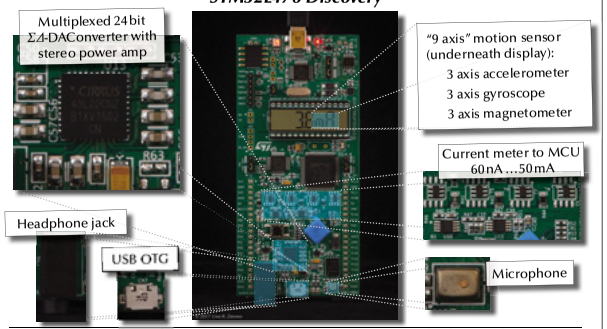
MCU

?



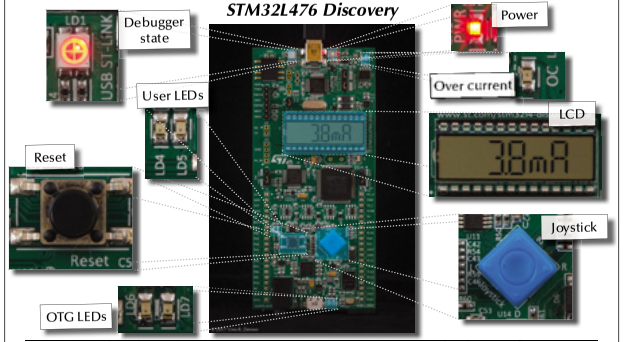
Asynchronism

STM32L476 Discovery



Asynchronism

STM32L476 Discovery



Asynchronism

STM32L476 Discovery

CPU

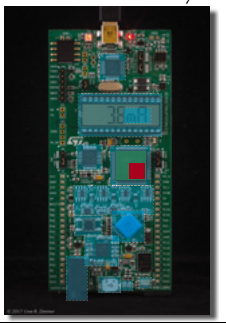
… running its sequence of machine instructions.

☞ How to interact with all the other devices inside the

MCU

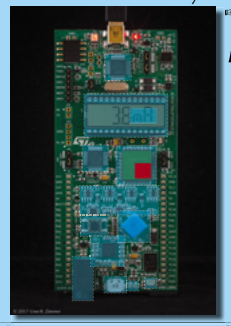
?

☞ … and then with all the devices on the board?



Asynchronism

STM32L476 Discovery



CPU

… running its sequence of machine instructions.

☞ How to interact with all the other devices inside the

MCU

?

☞ … and then with the devices on the board?

☞ and then with the

rest of the world

… which is connected to the board?

Asynchronism

Polling

Sequential machine instructions

☞ All external devices need to be "checked" by asking for their status.

☞ This should usually happen (semi-) regularly.

Asynchronism

Polling

Sequential machine instructions

☞ All external devices need to be "checked" by asking for their status.

☞ This should usually happen (semi-) regularly.

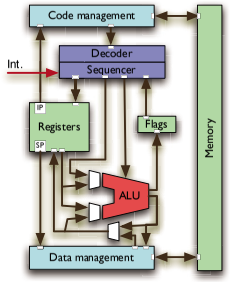
☞ This will lead to a loop of **polling** requests.

- ↗ Maximal latencies can be calculated straight forward.
- ↗ Simplicity of design (with small number of devices).
- ↗ Fastest option with small number of devices (like: one).
- ↘ All devices will need to wait their turn … even if this device is the only one with new data!
- ↘ The "main" program transforms into one large loop which can be hard to handle in terms of scalable program design.
- ↘ Events or data can be missed.

Asynchronism

Processor Architectures

Interrupts



- One or multiple lines wired directly into the sequencer

☞ *Required for*: **Pre-emptive scheduling**, **Timer driven actions**, **Transient hardware interactions**, …

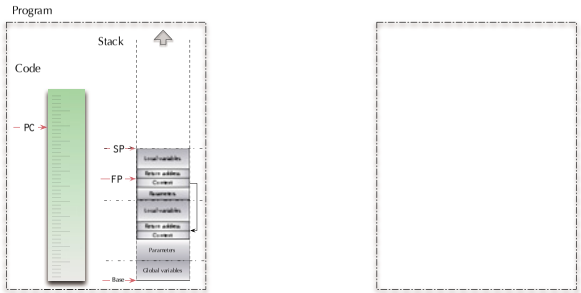
☞ Usually preceded by an external logic ("interrupt controller") which accumulates and encodes all external requests.

On interrupt (if unmasked):

- CPU stops normal sequencer flow.
- Lookup of interrupt handler's address
- Current IP and state pushed onto stack.
- IP set to interrupt handler.

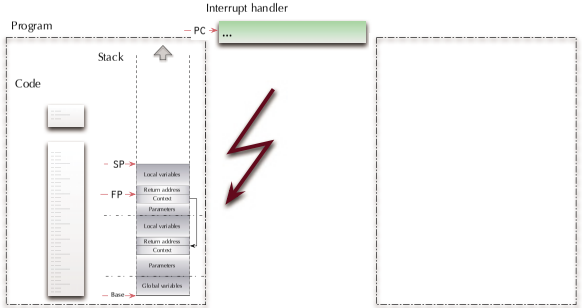
Asynchronism

Interrupt processing



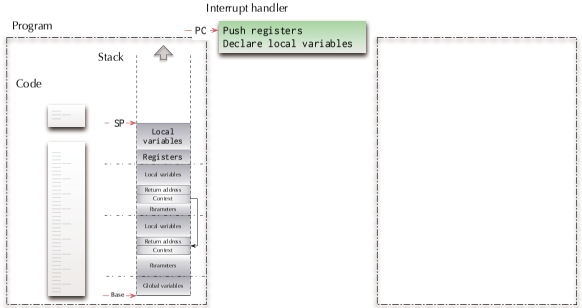
Asynchronism

Interrupt processing



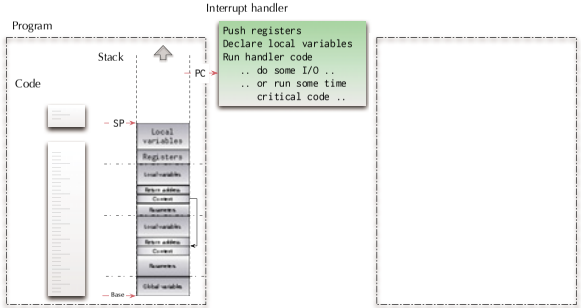
Asynchronism

Interrupt processing



Asynchronism

Interrupt processing



Asynchronism

Interrupt processing

Interrupt handler

Program

Stack

Code

PC

SP

Registers

Local variables

Return address

Context

Parameters

Local variables

Return address

Context

Parameters

Global variables

Base

```
Push registers
Declare local variables
Run handler code
   .. do some I/O ..
   .. or run some time
      critical code ..
Remove local variables
```

© 2019 Uwe R. Zimmer, The Australian National University

Asynchronism

Interrupt processing

Interrupt handler

Program

Stack

Code

PC

SP

FP

Local variables

Return address

Context

Parameters

Local variables

Return address

Context

Parameters

Global variables

Base

```
Push registers
Declare local variables
Run handler code
   .. do some I/O ..
   .. or run some time
      critical code ..
Remove local variables
Pop registers
```

© 2019 Uwe R. Zimmer, The Australian National University

Asynchronism

Interrupt processing

Interrupt handler

Program

Stack

Code

PC

SP

FP

Local variables

Return address

Context

Parameters

Local variables

Return address

Context

Parameters

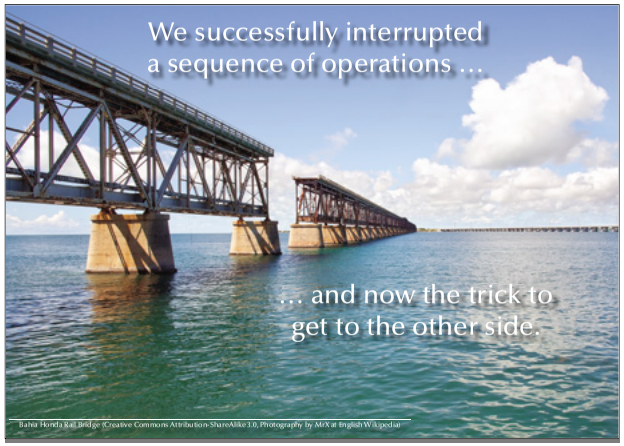
Global variables

Base

```
Push registers
Declare local variables
Run handler code
   .. do some I/O ..
   .. or run some time
      critical code ..
Remove local variables
Pop registers
```

Bahia Honda Rail Bridge (Creative Commons Attribution-ShareAlike 3.0, Photography by MrX at English Wikipedia)

© 2019 Uwe R. Zimmer, The Australian National University



Asynchronism

Interrupt processing

Interrupt handler

Program

Stack

Code

PC

SP

FP

Local variables

Return address

Context

Parameters

Local variables

Return address

Context

Parameters

Global variables

Base

© 2019 Uwe R. Zimmer, The Australian National University

Asynchronism

Interrupt processing

Interrupt handler

Program

Stack

Code

PC

```
...
```

SP

FP

Flags

PC

Local variables

Return address

Context

Parameters

Local variables

Return address

Context

Parameters

Global variables

Base

© 2019 Uwe R. Zimmer, The Australian National University

Asynchronism

Interrupt processing

Interrupt handler

Program

Stack

Code

PC

```
...
```

SP

FP

Flags

PC

Local variables

Return address

Context

Parameters

Local variables

Return address

Context

Parameters

Global variables

Base

The CPU hardware (!) did that, **before** anything was changed

© 2019 Uwe R. Zimmer, The Australian National University

Asynchronism

Interrupt processing

Interrupt handler

Program

Stack

Code

PC

```
Push registers
Declare local variables
```

SP

Local variables

Registers

Flags

PC

Local variables

Return address

Context

Parameters

Local variables

Return address

Context

Parameters

Global variables

Base

© 2019 Uwe R. Zimmer, The Australian National University

Asynchronism

Interrupt processing

Interrupt handler

Program

Stack

Code

PC

```
Push registers
Declare local variables
Run handler code
   .. do some I/O ..
   .. or run some time
      critical code ..
```

SP

Local variables

Registers

Flags

PC

Local variables

Return address

Context

Parameters

Local variables

Return address

Context

Parameters

Global variables

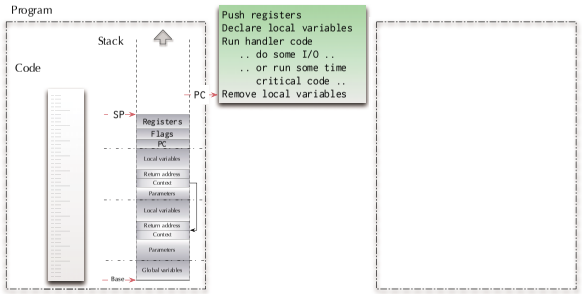
Base

© 2019 Uwe R. Zimmer, The Australian National University

Asynchronism

Interrupt processing

Interrupt handler

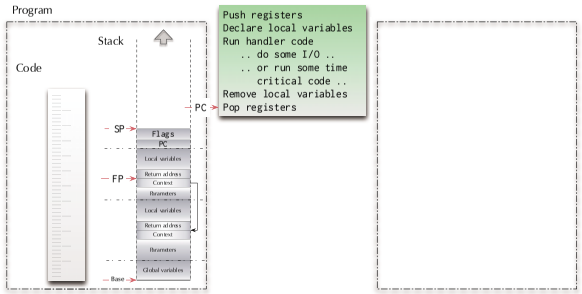



```
Push registers
Declare local variables
Run handler code
   .. do some I/O ..
   .. or run some time
      critical code ..
Remove local variables
```


Asynchronism

Interrupt processing

Interrupt handler

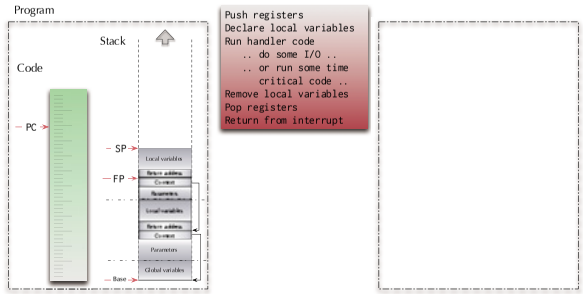



```
Push registers
Declare local variables
Run handler code
   .. do some I/O ..
   .. or run some time
      critical code ..
Remove local variables
Pop registers
```


Asynchronism

Interrupt processing

Interrupt handler

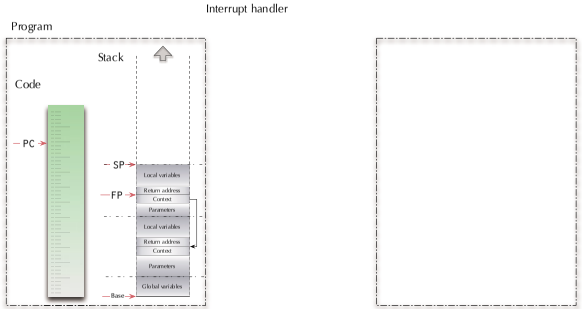



```
Push registers
Declare local variables
Run handler code
   .. do some I/O ..
   .. or run some time
      critical code ..
Remove local variables
Pop registers
Return from interrupt
```


Asynchronism

Interrupt processing

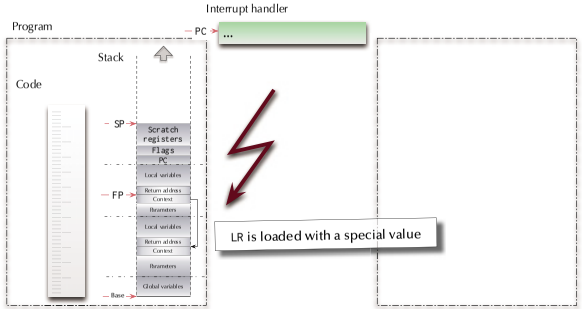
Interrupt handler



Asynchronism

Interrupt processing

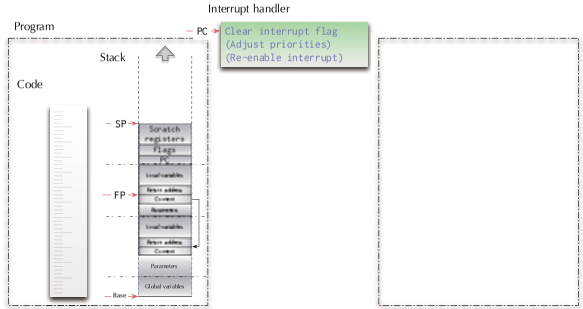
Interrupt handler



Asynchronism

Interrupt processing

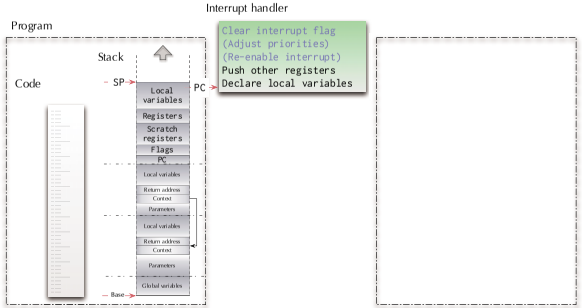
Interrupt handler



Asynchronism

Interrupt processing

Interrupt handler

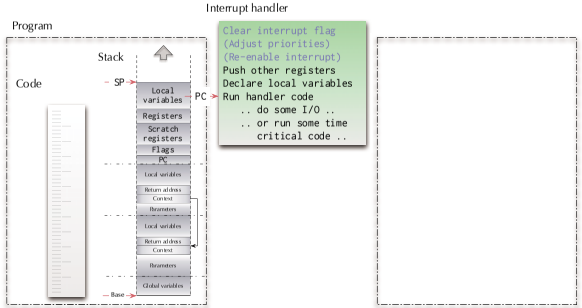



```
Clear interrupt flag
(Adjust priorities)
(Re-enable interrupt)
Push other registers
Declare local variables
```


Asynchronism

Interrupt processing

Interrupt handler

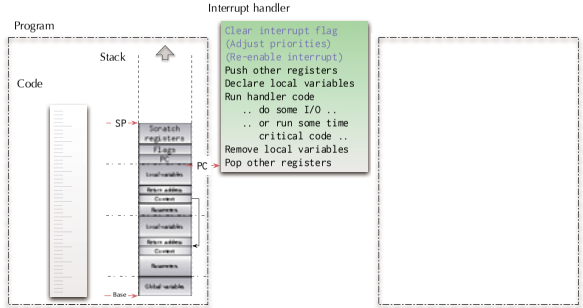



```
Clear interrupt flag
(Adjust priorities)
(Re-enable interrupt)
Push other registers
Declare local variables
Run handler code
   .. do some I/O ..
   .. or run some time
      critical code ..
```


Asynchronism

Interrupt processing

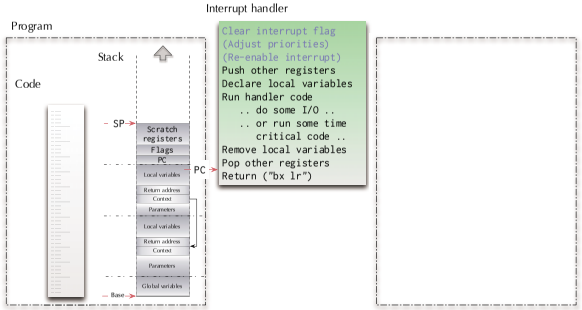
Interrupt handler




```
Clear interrupt flag
(Adjust priorities)
(Re-enable interrupt)
Push other registers
Declare local variables
Run handler code
   .. do some I/O ..
   .. or run some time
      critical code ..
Remove local variables
Pop other registers
```

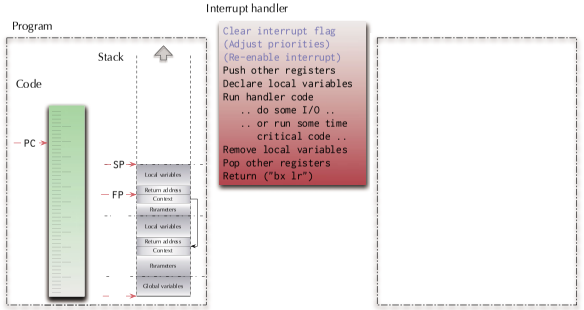
Asynchronism

Interrupt processing



Asynchronism

Interrupt processing



Asynchronism

Interrupt handler

Things to consider

- Interrupt handler code can be interrupted as well.
- Are you allowing to interrupt an interrupt handler with an interrupt on the same priority level (e.g. the same interrupt)?
- Can you overrun a stack with interrupt handlers?

Asynchronism

Interrupt handler

Things to consider

- Interrupt handler code can be interrupted as well.
- Are you allowing to interrupt an interrupt handler with an interrupt on the same priority level (e.g. the same interrupt)?
- Can you overrun a stack with interrupt handlers?
- Can we have one of those?

Asynchronism

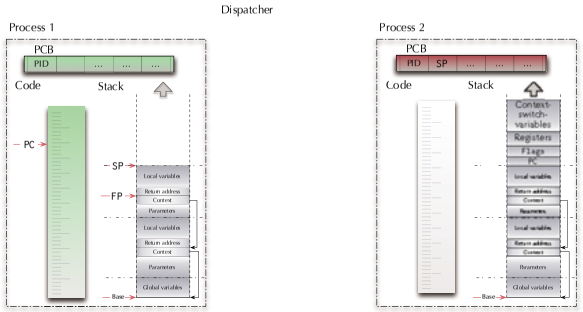
Multiple programs

If we can execute interrupt handler code "concurrently" to our "main" program:

- Can we then also have multiple "main" programs?

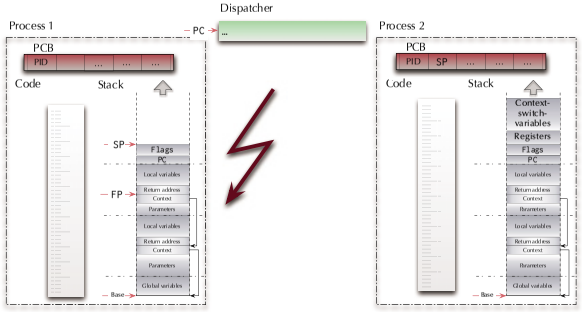
Asynchronism

Context switch



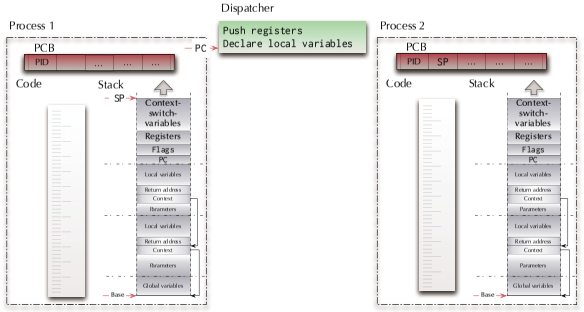
Asynchronism

Context switch



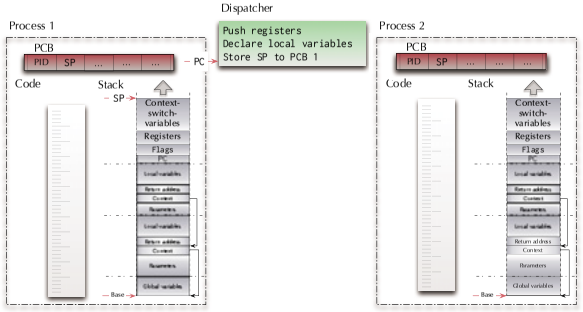
Asynchronism

Context switch



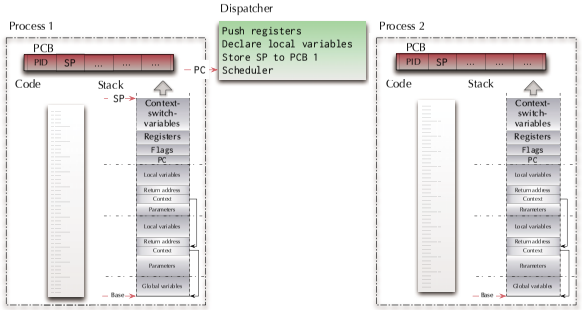
Asynchronism

Context switch



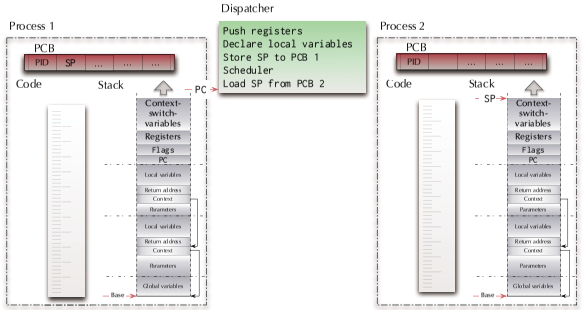
Asynchronism

Context switch



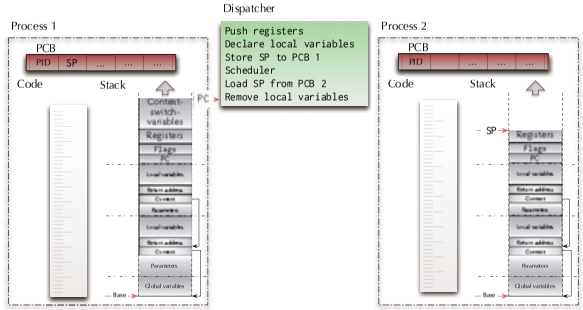
Asynchronism

Context switch



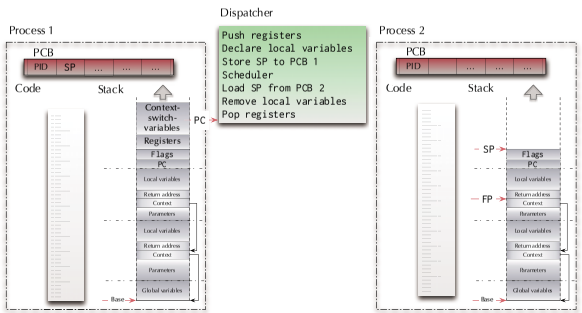
Asynchronism

Context switch



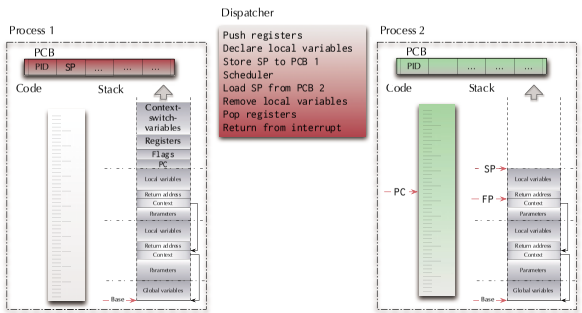
Asynchronism

Context switch



Asynchronism

Context switch



Asynchronism

Multi-tasking and Contention

Anything else could go wrong?

Asynchronism

Multi-tasking and Contention

Anything else could go wrong?

☞ If there is neither communication nor contention
between concurrent parts
… all is easy … and boring.

☞ What happens if concurrent programs **share** data?

Asynchronism

Shared variables

Atomic load & store operations

☞ Assumption 1: every individual base memory cell (word) load and store access is *atomic*
☞ Assumption 2: there is *no* atomic combined load-store access

```
G : Natural := 0; -- assumed to be mapped on a 1-word cell in memory
```

```
task body P1 is
begin
  G := 1
  G := G + G;
end P1;
```

```
task body P2 is
begin
  G := 2
  G := G + G;
end P2;
```

```
task body P3 is
begin
  G := 3
  G := G + G;
end P3;
```

☞ What is the value of G?

Asynchronism

Shared variables

Atomic load & store operations

☞ Assumption 1: every individual base memory cell (word) load and store access is *atomic*
☞ Assumption 2: there is *no* atomic combined load-store access

```
G: .word 0x00000000
```

```
ldr   r4, =G
mov   r1, #1
str   r1, [r4]
ldr   r2, [r4]
ldr   r3, [r4]
add   r1, r2, r3
str   r1, [r4]
```

```
ldr   r4, =G
mov   r1, #2
str   r1, [r4]
ldr   r2, [r4]
ldr   r3, [r4]
add   r1, r2, r3
str   r1, [r4]
```

```
ldr   r4, =G
mov   r1, #3
str   r1, [r4]
ldr   r2, [r4]
ldr   r3, [r4]
add   r1, r2, r3
str   r1, [r4]
```

☞ What is the value in memory cell G after all three programs complete?

Asynchronism

Shared variables

This is terrible!

Nobody is their right mind would analyse a program like that.

☞ ... are we missing something?

☞ ... is there an elegant way out?

Asynchronism

Mutual exclusion ... or the lack thereof

```
Count : Integer := 0;
```

```
task body Enter is
begin
 for i := 1 .. 100 loop
  Count := Count + 1;
 end loop;
end Enter;
```

```
task body Leave is
begin
 for i := 1 .. 100 loop
  Count := Count - 1;
 end loop;
end Leave;
```

☞ What is the value of Count after both programs complete?

Asynchronism

Mutual exclusion ... or the lack thereof

```
Count: .word 0x00000000

   ldr   r4, =Count
   mov   r1, #1
for_enter:
   cmp   r1, #100
   bgt   end_for_enter
   ldr   r2, [r4]
   add   r2, #1
   str   r2, [r4]
   add   r1, #1
   b     for_enter
end_for_enter:
```

```
   ldr   r4, =Count
   mov   r1, #1
for_leave:
   cmp   r1, #100
   bgt   end_for_leave
   ldr   r2, [r4]
   sub   r2, #1
   str   r2, [r4]
   add   r1, #1
   b     for_leave
end_for_leave:
```

☞ What is the value at address Count after both programs complete?

Asynchronism

Mutual exclusion ... or the lack thereof

```
Count: .word 0x00000000

   ldr   r4, =Count
   mov   r1, #1
for_enter:
   cmp   r1, #100
   bgt   end_for_enter
   ldr   r2, [r4]
   add   r2, #1
   str   r2, [r4]
   add   r1, #1
   b     for_enter
end_for_enter:
```

```
   ldr   r4, =Count
   mov   r1, #1
for_leave:
   cmp   r1, #100
   bgt   end_for_leave
   ldr   r2, [r4]
   sub   r2, #1
   str   r2, [r4]
   add   r1, #1
   b     for_leave
end_for_leave:
```

☞ What is the value at address Count after both programs complete?

Asynchronism

Mutual exclusion ... or the lack thereof

```
Count: .word 0x00000000

   ldr   r4, =Count
   mov   r1, #1
for_enter:
   cmp   r1, #100
   bgt   end_for_enter
enter_strex_fail:
   ldrex r2, [r4] ; tag [r4] as exclusive
   add   r2, #1
   strex r2, [r4] ; only if untouched
   cbnz  r2, enter_strex_fail
   add   r1, #1
   b     for_enter
end_for_enter:
```

```
   ldr   r4, =Count
   mov   r1, #1
for_leave:
   cmp   r1, #100
   bgt   end_for_leave
leave_strex_fail:
   ldrex r2, [r4] ; tag [r4] as exclusive
   sub   r2, #1
   strex r2, [r4] ; only if untouched
   cbnz  r2, leave_strex_fail
   add   r1, #1
   b     for_leave
end_for_leave:
```

☞ What is the value at address Count after both programs complete?

Asynchronism

Mutual exclusion

```
Count: .word 0x00000000

   ldr   r4, =Count
   mov   r1, #1
for_enter:
   cmp   r1, #100
   bgt   end_for_enter
enter_strex_fail:
   ldrex r2, [r4] ; tag [r4] as exclusive
   add   r2, #1
   strex r2, [r4] ; only if untouched
   cbnz  r2, enter_strex_fail
   add   r1, #1
   b     for_enter
end_for_enter:
```

```
   ldr   r4, =Count
   mov   r1, #1
for_leave:
   cmp   r1, #100
   bgt   end_for_leave
leave_strex_fail:
   ldrex r2, [r4] ; tag [r4] as exclusive
   sub   r2, #1
   strex r2, [r4] ; only if untouched
   cbnz  r2, leave_strex_fail
   add   r1, #1
   b     for_leave
end_for_leave:
```

Any context switch needs to clear reservations

☞ What is the value at address Count after both programs complete?


```
Count:   .word 0x00000000

   ldr   r4, =Count
   mov   r1, #1
for_enter:
   cmp   r1, #100
   bgt   end_for_enter
   ldr   r2, [r4]
   add   r2, #1
   str   r2, [r4]
   add   r1, #1
   b     for_enter
end_for_enter:
```

```
   ldr   r4, =Count
   mov   r1, #1
for_leave:
   cmp   r1, #100
   bgt   end_for_leave
   ldr   r2, [r4]
   sub   r2, #1
   str   r2, [r4]
   add   r1, #1
   b     for_leave
end_for_leave:
```

Negotiate who goes first

Critical section

Critical section

Indicate critical section completed


```
Count:   .word 0x00000000
Lock:    .word 0x00000000 ; #0 means unlocked
   ldr   r3, =Lock
   ldr   r4, =Count
   mov   r1, #1
for_enter:
   cmp   r1, #100
   bgt   end_for_enter
fail_enter:
   ldr   r0, [r3]
   cbnz  r0, fail_enter  ; if locked
   ldr   r2, [r4]
   add   r2, #1
   str   r2, [r4]
   add   r1, #1
   b     for_enter
end_for_enter:
```

```
   ldr   r3, =Lock
   ldr   r4, =Count
   mov   r1, #1
for_leave:
   cmp   r1, #100
   bgt   end_for_leave
fail_leave:
   ldr   r0, [r3]
   cbnz  r0, fail_leave  ; if locked
   ldr   r2, [r4]
   sub   r2, #1
   str   r2, [r4]
   add   r1, #1
   b     for_leave
end_for_leave:
```

Critical section

Critical section


```
Count:   .word 0x00000000
Lock:    .word 0x00000000 ; #0 means unlocked
   ldr   r3, =Lock
   ldr   r4, =Count
   mov   r1, #1
for_enter:
   cmp   r1, #100
   bgt   end_for_enter
fail_enter:
   ldr   r0, [r3]
   cbnz  r0, fail_enter  ; if locked
   mov   r0, #1          ; lock value
   str   r0, [r3]        ; lock
   ldr   r2, [r4]
   add   r2, #1
   str   r2, [r4]
   add   r1, #1
   b     for_enter
end_for_enter:
```

```
   ldr   r3, =Lock
   ldr   r4, =Count
   mov   r1, #1
for_leave:
   cmp   r1, #100
   bgt   end_for_leave
fail_leave:
   ldr   r0, [r3]
   cbnz  r0, fail_leave  ; if locked
   mov   r0, #1          ; lock value
   str   r0, [r3]        ; lock
   ldr   r2, [r4]
   sub   r2, #1
   str   r2, [r4]
   add   r1, #1
   b     for_leave
end_for_leave:
```

Critical section

Critical section

```
Count:   .word 0x00000000
Lock:    .word 0x00000000 ; #0 means unlocked
   ldr   r3, =Lock                    ldr   r3, =Lock
   ldr   r4, =Count                   ldr   r4, =Count
   mov   r1, #1                       mov   r1, #1
for_enter:                         for_leave:
   cmp   r1, #100                     cmp   r1, #100
   bgt   end_for_enter                bgt   end_for_leave
fail_enter:                        fail_leave:
   ldrex r0, [r3]                     ldrex r0, [r3]
   cbnz  r0, fail_enter  ; if locked  cbnz  r0, fail_leave  ; if locked
   mov   r0, #1          ; lock value mov   r0, #1          ; lock value
   strex r0, [r3]        ; try lock   strex r0, [r3]        ; try lock
   cbnz  r0, fail_enter  ; if touched cbnz  r0, fail_leave  ; if touched
   dmb                   ; sync memory dmb                  ; sync memory
   ldr   r2, [r4]                     ldr   r2, [r4]
   add   r2, #1                       sub   r2, #1
   str   r2, [r4]                     str   r2, [r4]
   dmb                   ; sync memory dmb                  ; sync memory
   mov   r0, #0          ; unlock value mov   r0, #0        ; unlock value
   str   r0, [r3]        ; unlock     str   r0, [r3]        ; unlock
   add   r1, #1                       add   r1, #1
   b     for_enter                    b     for_leave
end_for_enter:                     end_for_leave:
```

Any context switch needs to clear reservations

Critical section

Critical section


```
Count:   .word 0x00000000
Lock:    .word 0x00000000 ; #0 means unlocked
   ldr   r3, =Lock                    ldr   r3, =Lock
   ldr   r4, =Count                   ldr   r4, =Count
   mov   r1, #1                       mov   r1, #1
for_enter:                         for_leave:
   cmp   r1, #100                     cmp   r1, #100
   bgt   end_for_enter                bgt   end_for_leave
fail_enter:                        fail_leave:
   ldrex r0, [r3]                     ldrex r0, [r3]
   cbnz  r0, fail_enter  ; if locked  cbnz  r0, fail_leave  ; if locked
   mov   r0, #1          ; lock value mov   r0, #1          ; lock value
   strex r0, [r3]        ; try lock   strex r0, [r3]        ; try lock
   cbnz  r0, fail_enter  ; if touched cbnz  r0, fail_leave  ; if touched
   dmb                   ; sync memory dmb                  ; sync memory
   ldr   r2, [r4]                     ldr   r2, [r4]
   add   r2, #1                       sub   r2, #1
   str   r2, [r4]                     str   r2, [r4]
   dmb                   ; sync memory dmb                  ; sync memory
   mov   r0, #0          ; unlock value mov   r0, #0        ; unlock value
   str   r0, [r3]        ; unlock     str   r0, [r3]        ; unlock
   add   r1, #1                       add   r1, #1
   b     for_enter                    b     for_leave
end_for_enter:                     end_for_leave:
```

Any context switch needs to clear reservations

Critical section

Critical section

Asynchronism

Mutual exclusion: atomic test-and-set operation

```
type Flag is Natural range 0..1; C : Flag := 0;

task body Pi is                    task body Pj is
L : Flag;                          L : Flag;
begin                              begin
 loop                               loop
  loop                               loop
   [L := C; C := 1];                  [L := C; C := 1];
   exit when L = 0;                   exit when L = 0;
   ------ change process             ------ change process
  end loop;                          end loop;
  ------ critical_section_i;         ------ critical_section_j;
  C := 0;                            C := 0;
 end loop;                          end loop;
end Pi;                            end Pj;
```

☞ Does that work?

Asynchronism

Mutual exclusion: atomic test-and-set operation

```
type Flag is Natural range 0..1; C : Flag := 0;

task body Pi is                    task body Pj is
L : Flag;                          L : Flag;
begin                              begin
 loop                               loop
  loop                               loop
   [L := C; C := 1];                  [L := C; C := 1];
   exit when L = 0;                   exit when L = 0;
   ------ change process             ------ change process
  end loop;                          end loop;
  ------ critical_section_i;         ------ critical_section_j;
  C := 0;                            C := 0;
 end loop;                          end loop;
end Pi;                            end Pj;
```

☞ Mutual exclusion!, No deadlock!, No global live-lock!

☞ Works for any dynamic number of processes.

☞ Individual starvation possible! Busy waiting loops!

Asynchronism

Mutual exclusion: atomic exchange operation

```
type Flag is Natural range 0..1; C : Flag := 0;

task body Pi is                    task body Pj is
L : Flag := 1;                     L : Flag := 1;
begin                              begin
 loop                               loop
  loop                               loop
   [Temp := L; L := C; C := Temp];    [Temp := L; L := C; C := Temp];
   exit when L = 0;                   exit when L = 0;
   ------ change process             ------ change process
  end loop;                          end loop;
  ------ critical_section_i;         ------ critical_section_j;
  L := 1; C := 0;                    L := 1; C := 0;
 end loop;                          end loop;
end Pi;                            end Pj;
```

☞ Does that work?

Asynchronism

Mutual exclusion: atomic exchange operation

```
type Flag is Natural range 0..1; C : Flag := 0;

task body Pi is                    task body Pj is
L : Flag := 1;                     L : Flag := 1;
begin                              begin
 loop                               loop
  loop                               loop
   [Temp := L; L := C; C := Temp];    [Temp := L; L := C; C := Temp];
   exit when L = 0;                   exit when L = 0;
   ------ change process             ------ change process
  end loop;                          end loop;
  ------ critical_section_i;         ------ critical_section_j;
  L := 1; C := 0;                    L := 1; C := 0;
 end loop;                          end loop;
end Pi;                            end Pj;
```

☞ Mutual exclusion!, No deadlock!, No global live-lock!

☞ Works for any dynamic number of processes.

☞ Individual starvation possible! Busy waiting loops!

Asynchronism

Mutual exclusion: memory cell reservation

```
type Flag is Natural range 0..1; C : Flag := 0;

task body Pi is                    task body Pj is
L : Flag;                          L : Flag;
begin                              begin
 loop                               loop
  loop                               loop
   L :R= C; C :T= 1;                  L :R= C; C :T= 1;
   exit when Untouched and L = 0;     exit when Untouched and L = 0;
   ------ change process             ------ change process
  end loop;                          end loop;
  ------ critical_section_i;         ------ critical_section_j;
  C := 0;                            C := 0;
 end loop;                          end loop;
end Pi;                            end Pj;
```

☞ Does that work?

Asynchronism

Mutual exclusion: memory cell reservation

```
type Flag is Natural range 0..1; C : Flag := 0;

task body Pi is                    task body Pj is
L : Flag;                          L : Flag;
begin                              begin
 loop                               loop
  loop                               loop
   L :R= C; C :T= 1;                  L :R= C; C :T= 1;
   exit when Untouched and L = 0;     exit when Untouched and L = 0;
   ------ change process             ------ change process
  end loop;                          end loop;
  ------ critical_section_i;         ------ critical_section_j;
  C := 0;                            C := 0;
 end loop;                          end loop;
end Pi;                            end Pj;
```

Any context switch needs to clear reservations

☞ Mutual exclusion!, No deadlock!, No global live-lock!

☞ Works for any dynamic number of processes.

☞ Individual starvation possible! Busy waiting loops!


```
Count:   .word 0x00000000
Lock:    .word 0x00000000 ; #0 means unlocked
   ldr   r3, =Lock                    ldr   r3, =Lock
   ldr   r4, =Count                   ldr   r4, =Count
   mov   r1, #1                       mov   r1, #1
for_enter:                         for_leave:
   cmp   r1, #100                     cmp   r1, #100
   bgt   end_for_enter                bgt   end_for_leave
fail_enter:                        fail_leave:
   ldrex r0, [r3]                     ldrex r0, [r3]
   cbnz  r0, fail_enter  ; if locked  cbnz  r0, fail_leave  ; if locked
   mov   r0, #1          ; lock value mov   r0, #1          ; lock value
   strex r0, [r3]        ; try lock   strex r0, [r3]        ; try lock
   cbnz  r0, fail_enter  ; if touched cbnz  r0, fail_leave  ; if touched
   dmb                   ; sync memory dmb                  ; sync memory
   ldr   r2, [r4]                     ldr   r2, [r4]
   add   r2, #1                       sub   r2, #1
   str   r2, [r4]                     str   r2, [r4]
   dmb                   ; sync memory dmb                  ; sync memory
   mov   r0, #0          ; unlock value mov   r0, #0        ; unlock value
   str   r0, [r3]        ; unlock     str   r0, [r3]        ; unlock
   add   r1, #1                       add   r1, #1
   b     for_enter                    b     for_leave
end_for_enter:                     end_for_leave:
```

Any context switch needs to clear reservations

Asks for permission

Critical section

Critical section

Asynchronism

Mutual exclusion

```
Count: .word 0x00000000

   ldr   r4, =Count                    ldr   r4, =Count
   mov   r1, #1                        mov   r1, #1
for_enter:                          for_leave:
   cmp   r1, #100                      cmp   r1, #100
   bgt   end_for_enter                 bgt   end_for_leave
enter_strex_fail:                   leave_strex_fail:
   ldrex r2, [r4] ; tag [r4] as exclusive   ldrex r2, [r4] ; tag [r4] as exclusive
   add   r2, #1                        sub   r2, #1
   strex r2, [r4] ; only if untouched  strex r2, [r4] ; only if untouched
   cbnz  r2, enter_strex_fail          cbnz  r2, leave_strex_fail
   add   r1, #1                        add   r1, #1
   b     for_enter                     b     for_leave
end_for_enter:                      end_for_leave:
```

Any context switch needs to clear reservations

Asks for forgiveness

☞ Light weight solution – sometimes referred to as "lock-free" or "lockless".

Asynchronism

Beyond atomic hardware operations

Semaphores

Basic definition (Dijkstra 1968)

Assuming the following three conditions on a shared memory cell between processes:

- a set of processes agree on a variable **S** operating as a flag to indicate synchronization conditions
- an atomic operation **P** on S — for 'passeren' (Dutch for 'pass'):
  P(S): `[as soon as S > 0 then S := S - 1]` ☞ this is a potentially delaying operation
- an atomic operation **V** on S — for 'vrygeven' (Dutch for 'to release'):
  V(S): `[S := S + 1]`

☞ *then* the variable **S** is called a **Semaphore**.

Asynchronism

Beyond atomic hardware operations

Semaphores

… as supplied by operating systems and runtime environments

- a set of processes $P_1 \ldots P_N$ agree on a variable **S** operating as a flag to indicate synchronization conditions
- an atomic operation **Wait** on S: (aka 'Suspend_Until_True', 'sem_wait', …)
  Process P_i : **Wait** (S):
  `[if S > 0 then S := S - 1`
  `else suspend` P_i `on S]`
- an atomic operation **Signal** on S: (aka 'Set_True', 'sem_post', …)
  Process P_i : **Signal** (S):
  `[if` $\exists P_j$ `suspended on S then release` P_j
  `else S := S + 1]`

☞ *then* the variable **S** is called a **Semaphore** in a scheduling environment.

Asynchronism

Beyond atomic hardware operations

Semaphores

Types of semaphores:

- **Binary semaphores**: restricted to [0, 1] or [False, True] resp.
  Multiple `V` (`Signal`) calls have the same effect than a single call.
  - Atomic hardware operations support binary semaphores.
  - Binary semaphores are sufficient to create all other semaphore forms.
- **General semaphores** (counting semaphores): non-negative number; (range limited by the system) `P` and `V` increment and decrement the semaphore by one.
- **Quantity semaphores**: The increment (and decrement) value for the semaphore is specified as a parameter with P and V.

☞ All types of semaphores must be initialized:
often the number of processes which are allowed inside a critical section, i.e. '1'.


```
Count:   .word 0x00000000
Sema:    .word 0x00000001

   ldr   r3, =Sema                     ldr   r3, =Sema
   ldr   r4, =Count                    ldr   r4, =Count
   mov   r1, #1                        mov   r1, #1
for_enter:                          for_leave:
   cmp   r1, #100                      cmp   r1, #100
   bgt   end_for_enter                 bgt   end_for_leave
wait_1:                             wait_2:
   ldr   r0, [r3]                      ldr   r0, [r3]
   cbz   r0, wait_1 ; if Semaphore = 0 cbz   r0, wait_2 ; if Semaphore = 0
   …                                   …
                                       Critical section
   add   r1, #1                        add   r1, #1
   b     for_enter                     b     for_leave
end_for_enter:                      end_for_leave:
```



```
Count:   .word 0x00000000
Sema:    .word 0x00000001

   ldr   r3, =Sema                     ldr   r3, =Sema
   ldr   r4, =Count                    ldr   r4, =Count
   mov   r1, #1                        mov   r1, #1
for_enter:                          for_leave:
   cmp   r1, #100                      cmp   r1, #100
   bgt   end_for_enter                 bgt   end_for_leave
wait_1:                             wait_2:
   ldr   r0, [r3]                      ldr   r0, [r3]
   cbz   r0, wait_1 ; if Semaphore = 0 cbz   r0, wait_2 ; if Semaphore = 0
   sub   r0, #1     ; dec Semaphore    sub   r0, #1     ; dec Semaphore
   str   r0, [r3]   ; update          str   r0, [r3]   ; update
   …                                   …
                                       Critical section
   add   r1, #1                        add   r1, #1
   b     for_enter                     b     for_leave
end_for_enter:                      end_for_leave:
```



```
Count:   .word 0x00000000
Sema:    .word 0x00000001

   ldr   r3, =Sema                     ldr   r3, =Sema
   ldr   r4, =Count                    ldr   r4, =Count
   mov   r1, #1                        mov   r1, #1
for_enter:                          for_leave:
   cmp   r1, #100                      cmp   r1, #100
   bgt   end_for_enter                 bgt   end_for_leave
wait_1:                             wait_2:
   ldrex r0, [r3]                      ldrex r0, [r3]
   cbz   r0, wait_1 ; if Semaphore = 0 cbz   r0, wait_2 ; if Semaphore = 0
   sub   r0, #1     ; dec Semaphore    sub   r0, #1     ; dec Semaphore
   strex r0, [r3]   ; try update      strex r0, [r3]   ; try update
   cbnz  r0, wait_1 ; if touched      cbnz  r0, wait_2 ; if touched
   dmb              ; sync memory     dmb              ; sync memory
   …                                   …
                                       Critical section
   add   r1, #1                        add   r1, #1
   b     for_enter                     b     for_leave
end_for_enter:                      end_for_leave:
```

Any context switch needs to clear reservations


```
Count:   .word 0x00000000
Sema:    .word 0x00000001

   ldr   r3, =Sema                     ldr   r3, =Sema
   ldr   r4, =Count                    ldr   r4, =Count
   mov   r1, #1                        mov   r1, #1
for_enter:                          for_leave:
   cmp   r1, #100                      cmp   r1, #100
   bgt   end_for_enter                 bgt   end_for_leave
wait_1:                             wait_2:
   ldrex r0, [r3]                      ldrex r0, [r3]
   cbz   r0, wait_1 ; if Semaphore = 0 cbz   r0, wait_2 ; if Semaphore = 0
   sub   r0, #1     ; dec Semaphore    sub   r0, #1     ; dec Semaphore
   strex r0, [r3]   ; try update      strex r0, [r3]   ; try update
   cbnz  r0, wait_1 ; if touched      cbnz  r0, wait_2 ; if touched
   dmb              ; sync memory     dmb              ; sync memory
   …                                   …
                                       Critical section
   ldr   r0, [r3]                      ldr   r0, [r3]
   add   r0, #1       ; inc Semaphore  add   r0, #1       ; inc Semaphore
   str   r0, [r3]     ; update         str   r0, [r3]     ; update
   add   r1, #1                        add   r1, #1
   b     for_enter                     b     for_leave
end_for_enter:                      end_for_leave:
```

Any context switch needs to clear reservations


```
Count:   .word 0x00000000
Sema:    .word 0x00000001

   ldr   r3, =Sema                     ldr   r3, =Sema
   ldr   r4, =Count                    ldr   r4, =Count
   mov   r1, #1                        mov   r1, #1
for_enter:                          for_leave:
   cmp   r1, #100                      cmp   r1, #100
   bgt   end_for_enter                 bgt   end_for_leave
wait_1:                             wait_2:
   ldrex r0, [r3]                      ldrex r0, [r3]
   cbz   r0, wait_1 ; if Semaphore = 0 cbz   r0, wait_2 ; if Semaphore = 0
   sub   r0, #1     ; dec Semaphore    sub   r0, #1     ; dec Semaphore
   strex r0, [r3]   ; try update      strex r0, [r3]   ; try update
   cbnz  r0, wait_1 ; if touched      cbnz  r0, wait_2 ; if touched
   dmb              ; sync memory     dmb              ; sync memory
   …                                   …
                                       Critical section
signal_1:                           signal_2:
   ldrex r0, [r3]                      ldrex r0, [r3]
   add   r0, #1       ; inc Semaphore  add   r0, #1       ; inc Semaphore
   strex r0, [r3]     ; try update     strex r0, [r3]     ; try update
   cbnz  r0, signal_1 ; if touched     cbnz  r0, signal_2 ; if touched
   dmb                ; sync memory    dmb                ; sync memory
   add   r1, #1                        add   r1, #1
   b     for_enter                     b     for_leave
end_for_enter:                      end_for_leave:
```

Any context switch needs to clear reservations

Asynchronism

Semaphores

```
S : Semaphore := 1;

task body Pi is
begin
 loop
  ------ non_critical_section_i;
  wait (S);
   ------ critical_section_i;
  signal (S);
 end loop;
end Pi;

task body Pj is
begin
 loop
  ------ non_critical_section_j;
  wait (S);
   ------ critical_section_j;
  signal (S);
 end loop;
end Pi;
```

☞ Works?

Asynchronism

Semaphores

```
S : Semaphore := 1;

task body Pi is
begin
 loop
  ------ non_critical_section_i;
  wait (S);
   ------ critical_section_i;
  signal (S);
 end loop;
end Pi;

task body Pj is
begin
 loop
  ------ non_critical_section_j;
  wait (S);
   ------ critical_section_j;
  signal (S);
 end loop;
end Pi;
```

☞ Mutual exclusion!, No deadlock!, No global live-lock!
☞ Works for any dynamic number of processes
☞ Individual starvation possible!

Asynchronism

Semaphores

```
S1, S2 : Semaphore := 1;

task body Pi is
begin
 loop
  ------ non_critical_section_i;
  wait (S1);
  wait (S2);
   ------ critical_section_i;
  signal (S2);
  signal (S1);
 end loop;
end Pi;

task body Pj is
begin
 loop
  ------ non_critical_section_j;
  wait (S2);
  wait (S1);
   ------ critical_section_j;
  signal (S1);
  signal (S2);
 end loop;
end Pi;
```

☞ Works too?

Asynchronism

Semaphores

```
S1, S2 : Semaphore := 1;

task body Pi is
begin
 loop
  ------ non_critical_section_i;
  wait (S1);
  wait (S2);
   ------ critical_section_i;
  signal (S2);
  signal (S1);
 end loop;
end Pi;

task body Pj is
begin
 loop
  ------ non_critical_section_j;
  wait (S2);
  wait (S1);
   ------ critical_section_j;
  signal (S1);
  signal (S2);
 end loop;
end Pi;
```

☞ Mutual exclusion!, No global live-lock!
☞ Works for any dynamic number of processes.
☞ Individual starvation possible!
☞ Deadlock possible!

Asynchronism

Semaphores

```
S1, S2 : Semaphore := 1;

task body Pi is
begin
 loop
  ------ non_critical_section_i;
  wait (S1);
  wait (S2);
   ------ critical_section_i;
  signal (S2);
  signal (S1);
 end loop;
end Pi;

task body Pj is
begin
 loop
  ------ non_critical_section_j;
  wait (S2);
  wait (S1);
   ------ critical_section_j;
  signal (S1);
  signal (S2);
 end loop;
end Pi;
```

☞ Mutual exclusion!, No global live-lock!
☞ Works for any dynamic number of processes.
☞ Individual starvation possible!
☞ Deadlock possible!

Concurrent programming languages offer **higher abstraction** and **safer** synchronization mechanisms.

Asynchronism

Summary

Aynchronism

- **Interrupts & Exceptions**
  - Concept
  - Hardware/Software interaction
  - Recursive interrupts
- **Concurrency & Synchronization**
  - Race conditions
  - Synchronization
  - Passing data